



Week1_예습과제_김재희

ResNet - Deep Residual Learning for Image Recognition

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work

3. 제안 방법론

Main Idea

3.1. Residual Learning

3.2. Identity Mapping by Shortcuts

Contribution

3.3. Network Architectures

3.4. Implementation

4. 실험 및 결과

4.1. ImageNet Classification

Dataset

Baseline

결과

4.2. CIFAR-10 and Analysis

Dataset

Baseline

결과

5. 결론 (배운점)

1. Introduction

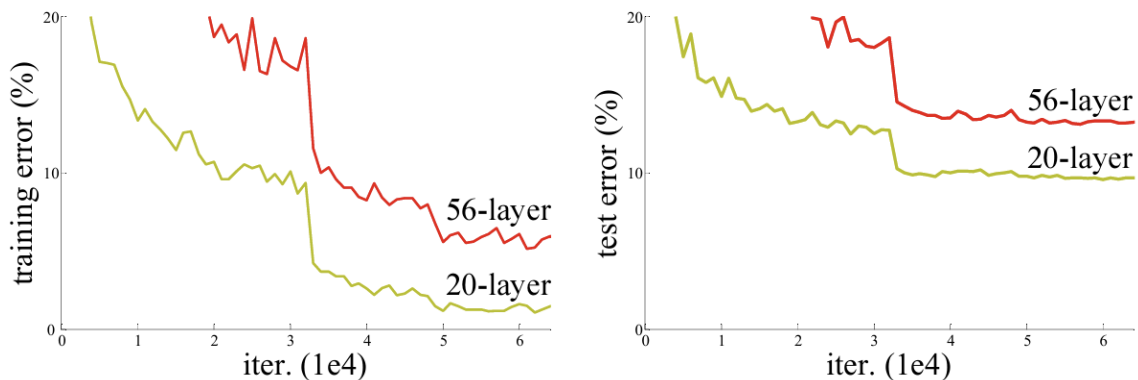


논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- Classification
- Localization
- Detection
- Segmentation

해당 task에서 기존 연구 한계점



CNN에서 층을 여러 개 쌓는 것은 곧 더 나은 성능을 의미했다. 하지만 과하게 층을 쌓았더니 오히려 층을 덜 쌓는 것보다 성능 저하가 일어났다. 연구진들은 이 문제의 원인을 파악하기 위해 얇은 모델에 추가된 레이어들을 항등 매핑하여 깊은 모델을 만들고 차이가 있는지 확인했다. 결과적으로는 더 깊은 모델의 성능이 하락했다. 이는 SGD(solver)가 더 깊은 모델에선 해를 찾기가 어렵다는 것을 시사한다.

논문의 contributions

- ImageNet 테스트 세트에서 3.57%의 Top-5 에러율을 기록하여 ILSVRC 2015 분류 대회에서 1위 차지
- ILSVRC 및 COCO 2015 대회의 ImageNet 탐지(detection), ImageNet 위치 특정(localization), COCO 탐지(detection), COCO 분할(segmentation) 부문에서도 모두 1위 차지
- 잔차 학습 원리가 일반적으로 적용 가능함을 보임

- Vision뿐만 아니라 non-vision 문제에도 적용 가능할 것이라 기대됨

2. Related Work



Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

Residual Representations

원본 값을 직접 다루는 것보다 잔차를 다루는 것이 여러 분야에서 이미 더 효과적이라고 알려져 있었다.

- VLAD / Fisher Vector: 이미지 검색 및 분류에 쓰이던 기법들이다. 이미지를 표현할 때 사전에 정해둔 기준점에서 잔차가 어떻게 되는지 인코딩하는 방식이다. 원본 데이터를 그대로 쓰는 것이 아니라 기준점 대비 차이로 표현했더니 성능이 좋았다.
- 벡터 양자화: 데이터 압축 및 표현 기법이다. 원본 벡터를 그냥 인코딩하는 것보다 잔차 벡터를 인코딩하는 것이 더 효과적이었다.
- 다중 격자법: 편미분방정식을 풀 때 쓰는 수치해석 기법이다. 복잡한 문제를 한 번에 풀지 않고, 큰 스케일과 작은 스케일로 나눠서 각 단계마다 이전 근사값과 정답의 차이(잔차)만 계산하는 방식이다. 원본 문제를 통째로 푸는 표준적인 방법보다 훨씬 빠르게 수렴한다고 알려져 있다.
- 계층적 기저 사전 조건화: 두 스케일 사이의 잔차를 변수로 쓴다.

→ 문제를 풀기 쉬운 형태(잔차를 다루는 방식)로 바꾸면 최적화 난이도가 확 낮아진다. 그래서 ResNet에서도 같은 기법을 적용해 보고자 했다.

Shortcut Connections

레이어를 건너뛰어 연결하는 shortcut 아이디어는 이미 알려져 있었다.

- 초창기 MLP: 입력을 출력에 바로 연결하는 선형 레이어를 추가한 사례가 있다.
- 보조 분류기: 기울기 소실 문제를 해결하기 위해 중간 레이어를 보조 분류기에 직접 연결한 사례가 있다.
- Centering: 레이어의 응답이나 기울기의 평균을 0 근처로 맞추기 위해 shortcut이 사용된 사례가 있다.
- 인셉션 구조에서 shortcut 브랜치가 쓰인 사례가 있다.

Highway Networks라는 shortcut 사용한 사례가 있다. 해당 논문에서는 이 네트워크와 ResNet의 차이를 서술한다.

	Highway Networks	ResNet
게이트 종류	게이팅 함수 O (파라미터 존재 → 데이터에 따라 달라짐)	항등 숏컷 (파라미터 X, 무조건 고정)
게이트 닫히는 경우	해당 레이어는 비잔차 함수가 되어 일반 레이어처럼 동작	닫히는 경우 X
정보 흐름	게이트가 닫히면 숏컷 경로 막힘	닫히는 경우가 없어서 숏컷도 안 막힘. 항상 원본 정보 그대로 통과됨.
100층 이상에서의 성과	정확도 향상 입증 X	152층까지 성공적으로 성능 향상 입증

3. 제안 방법론



Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

3.1. Residual Learning

신경망 레이어들은 대부분의 복잡한 함수는 모두 근사할 수 있다. 따라서 레이어들이 $H(x)$ 라는 비교적 복잡한 함수를 근사할 수 있다면 $H(x)-x$ 또한 근사할 수 있어야 한다. 만약 항등 매핑($H(x)=x$)이 최적의 답일 때, $H(x)$ 를 직접 학습하는 방식으로는 SGD가 최적 해를 잘 찾지 못한다. 하지만 잔차 방식($F(x)=H(x)-x$)이라면 $F(x)=x-x=0$, 즉 레이어 가중치를 0으로 몰아가면 되므로 잔차 방식이 훨씬 해를 구하기 쉽다. 대부분의 함수 $H(x)$ 는 완전히 0을 출력하는 것보다는 입력 x 를 어느 정도 유지하면서 조금씩만 바꾸는 것에 더 가깝다. 만약 $H(x)$ 를 잔차 방식으로 배우면 $F(x)$, 즉 원하는 값에서 x 를 뺀 만큼의 작은 조정값만 학습하면 된다. 기존 $H(x)$ 를 통째로 구하는 방식을 쓴다면 매번 처음부터 $H(x)$ 를 다시 구성해야 한다. 실제로 $F(x)$ 값들은 대체로 작은 값들이었다.

3.2. Identity Mapping by Shortcuts

$$y = F(x, \{W_i\}) + x$$

x : building block에 들어오는 입력

y : 해당 block의 최종 출력

$F(x, \{W_i\})$: 레이어들이 실제로 학습하는 잔차 함수. $\{W_i\}$ 는 해당 레이어들의 가중치들

숏컷 연결에는 파라미터가 추가되지 않는다. x 를 더하는 건 덧셈일 뿐이다. 덕분에 파라미터 개수, 깊이, 폭, 연산량이 똑같은 plain 네트워크와 residual 네트워크를 동등한 조건에서 비교 가능하다.

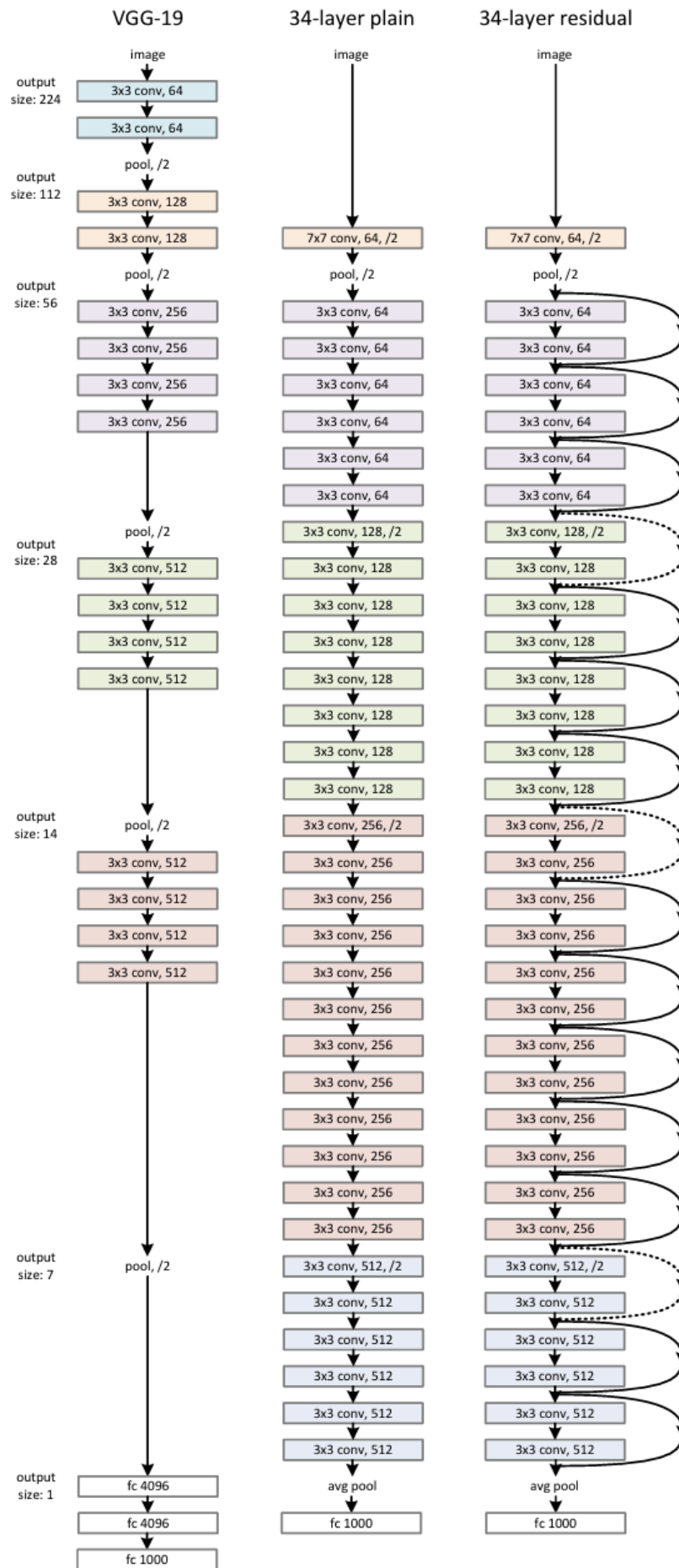
식이 성립하려면 x 와 $F(x)$ 의 차원이 같아야 한다. 하지만 차원이 다른 부분이 있다면 x 에 W_s 라는 행렬을 곱해서 $F(x)$ 와 같은 차원으로 맞춰준 다음 더해야 한다. 이 과정은 선택적이다.

F 가 단일 레이어일 때는 residual 네트워크에 별다른 이점이 없었다. F 가 2~3개 레이어로 구성된 경우에만 잔차 블록이 의미가 있었다.

위 설명은 모두 fully-connected 레이어를 기준으로 하지만 convolutional 레이어에도 똑같이 적용할 수 있다. 덧셈은 feature map끼리 채널별로 이루어진다.

Contribution

3.3. Network Architectures



34층 Plain Network: 슛컷 없이 레이어만 쌓은 대조군. feature map 크기가 절반으로 줄어들면 필터 개수를 2배로 늘렸다. 이렇게 하면 레이어당 계산량이 어느 정도 비슷하게 유지된다. 해당 네트워크는 VGG-19보다 필터 수가 적고 복잡도가 낮다. 즉 층은 더 많지만 실제 연산량은 훨씬 적은 네트워크이다.

34층 Residual Network: 위 Plain Network에 슛컷 연결을 추가한 것. 나머지 구조는 전부 같다.

실선 슛컷: 입력 x 와 출력 $F(x)$ 의 채널 수가 같은 경우 $\rightarrow$ 항등 매핑

점선 슛컷: feature map 크기가 줄어들면서 채널 수가 늘어나는 지점. 차원이 맞지 않기 때문에 그냥 더할 수 없다.

Residual Network이 차원이 맞지 않는 구간 처리하는 방법

1. Zero-padding

- 늘어난 채널만큼 0을 채워 넣어서 차원을 맞춤
- 추가 파라미터가 없음

2. Projection

- 1×1 conv를 이용한 projection으로 차원 맞춤
- 학습 가능한 파라미터 추가됨

3.4. Implementation

Training 세팅

1. 스케일 증강: 짧은 변을 [256, 480] 중 랜덤 샘플링

- 이미지를 리사이즈할 때 짧은 쪽 변 길이를 256~480 픽셀 사이에서 랜덤하게 정함
- 매번 다른 스케일로 리사이즈하면 모델이 크기 변화에 덜 민감하게 학습됨

2. 224×224 랜덤 크롭 + 수평 뒤집기

- 리사이즈된 이미지에서 224×224 크기만큼 랜덤 크롭. 또는 그 이미지를 좌우로 뒤집은 버전에서 크롭.

3. per-pixel mean subtracted

- 전체 학습 데이터셋의 픽셀별 평균값 미리 계산해두고 각 이미지에서 이 평균을 빼줌
 - 데이터를 0 근처로 중심화시켜서 학습을 더 안정적으로 만들
4. color augmentation
 - 밝기, 대비, 채도 등을 약간씩 변형하여 증강
 5. Batch Normalization
 - conv → BN → ReLU
 6. 가중치 초기화
 - He initialization: ReLU 쓰는 깊은 네트워크에 적합한 초기화 방법
 - 모든 모델을 처음부터 학습함. 사전학습된 가중치 안 씀.
 7. 최적화 하이퍼파라미터
 - SGD, mini-batch size 256
 - learning rate: 0.1, 성능 더 안 좋아질 때마다 학습률을 10으로 나눔
 - 최대 60만 iteration까지 학습
 - weight decay: 0.0001
 - momentum: 0.9
 8. Dropout
 - 안 쓰임
 - BN을 이미 쓰고 있어서 Dropout은 따로 안 씀

Testing 세팅

1. 10-crop testing
 - 테스트 이미지 하나에서 10개의 서로 다른 crop을 뽑아 모델에 넣고, 나온 결과들을 종합해서 평균낸 것
 - 여러 crop 예측을 평균내면 단일 crop보다 더 안정적이고 정확한 예측이 나옴
2. Fully-convolutional + multi-scale averaging
 - 최고 점수를 내기 위해 FC 레이어 대신 fully-convolutional 형태를 씀. 입력 이미지 크기에 제약이 없어서 원본 이미지를 crop하지 않고 통째로 넣을 수 있음

- 이미지를 5가지 다른 크기로 리사이즈하여 모델에 각각 넣고 나온 점수들을 평균냄
- 여러 스케일에서 얻은 예측을 종합하면 크기에 덜 민감하고 더 robust한 성능을 얻을 수 있음

4. 실험 및 결과



Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

4.1. ImageNet Classification

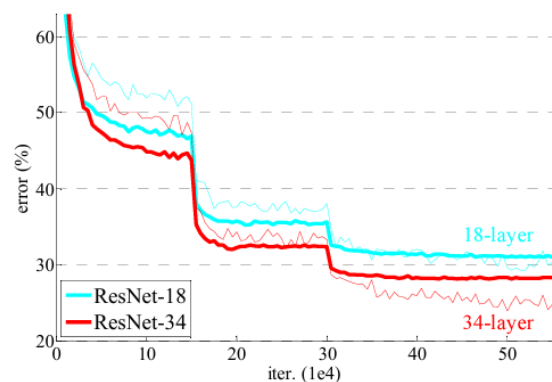
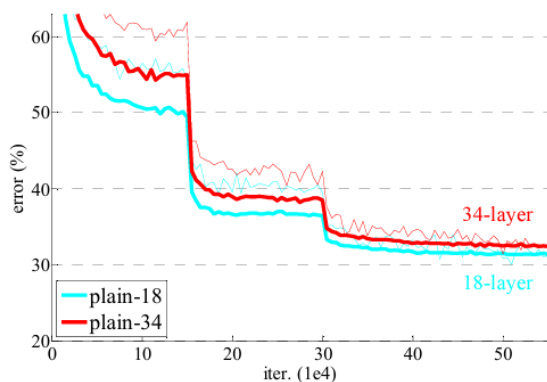
Dataset

ImageNet 2012

- 1000개 클래스
- 학습 데이터 128만 장
- 검증 데이터 5만 장
- 테스트 데이터 10만 장

Baseline

Plain Networks



plain network는 34층이 18층보다 에러율이 더 높다. → degradation problem 증거

ResNet 같은 경우엔 34층이 18층보다 성능이 좋다. 숏컷을 추가해서 깊이가 늘어난 만큼 성능이 좋아진 것이다.

residual network는 plain network과 비교했을 때 추가 파라미터가 전혀 없다. 즉 zero-padding 방식을 사용한 것. 따라서 성능 차이가 순전히 숏컷 때문이라고 말할 수 있다.

또한 성능 저하는 기울기 소실 때문이 아니다. 배치 정규화가 사용되고 있고, 역전파 기울기를 확인했을 때 정상적인 크기를 유지했다. 이 때문에 깊은 plain network에서는 수렴 속도가 매우 느리기 때문에 성능이 저하되었으리라고 짐작했다.

학습 iteration을 3배로 늘렸지만 여전히 성능 저하 문제가 관찰됐다. 단순 학습 시간을 늘리는 것만으로 해결되는 문제가 아님을 나타낸다.

18층에서는 plain net과 ResNet 정확도가 비슷하지만 ResNet이 더 빠르게 수렴했다. ResNet이 학습 초기 단계에서 훨씬 빠르게 수렴하는 이점 또한 있음을 보여준다.

method	top-1 err.	top-5 err.
VGG [40] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [43] (ILSVRC'14)	-	7.89
VGG [40] (v5)	24.4	7.1
PReLU-net [12]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

ResNet을 VGG, GoogLeNet, PReLU-net, BN-inception 등 다른 모델들과 비교했을 때, ResNet-152가 top-1 에러와 top-5 에러에서 에러율이 가장 낮아 성능이 가장 좋게 나왔다.

결과

Residual Networks

Plain network와 똑같은 구성에 숏컷만 추가되었다.

model	top-1 err.	top-5 err.
VGG-16 [40]	28.07	9.33
GoogLeNet [43]	-	9.15
PReLU-net [12]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

A: zero- padding

B: 차원이 늘어나는 지점에 projection 적용, 나머지는 순수 identity

C: 모든 숏컷에 projection 적용

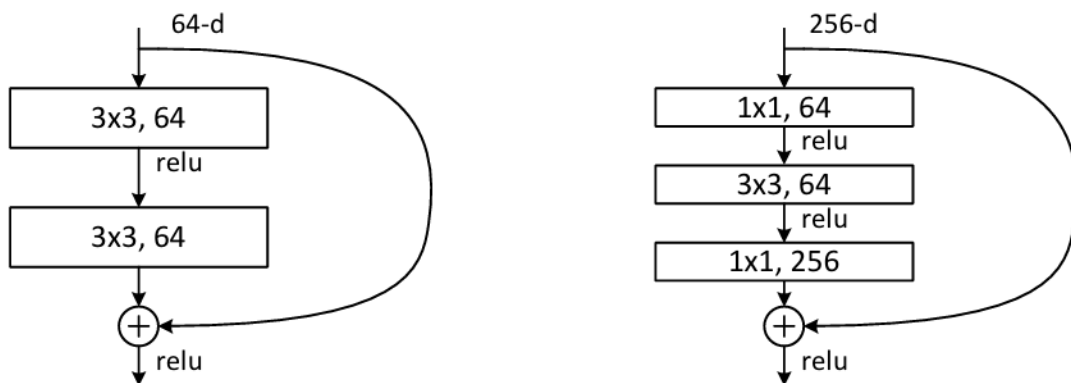
결과적으로 C, B, A순으로 성능이 좋았다. B가 A보다 나은 이유는 A의 zero-padded된 부분들은 residual learning을 전혀 하지 않기 때문에 그만큼 표현력이 손실되었기 때문이다. C가 B보다 나은 이유는 projection 덕분에 표현력이 조금 더 생겼기 때문이다. 하지만 차이가 미미하기 때문에 C처럼 모든 곳에 파라미터를 추가하는 것이 성능 저하 문제를 해결하는데 필수는 아니다. 즉, A처럼 순수 항등 매핑만으로도 충분히 효과가 있다. 이후에는 C를 쓰지 않기로 결정한다. 파라미터가 많을수록 메모리 사용량이나 시간 복잡도가 증가하기 때문이다. 또 뒤에 서술할 bottleneck 구조에서는 파라미터가 없는 identity shortcut 방식을 쓰는 것이 복잡도를 늘리지 않기 때문이다.

또 ResNet 깊이가 깊어질수록 어려움이 계속 낮아진다. Plain Net이었다면 깊이가 깊어질수록 성능이 좋아지지 않는 것을 알 수 있다.

method	top-5 err. (test)
VGG [40] (ILSVRC'14)	7.32
GoogLeNet [43] (ILSVRC'14)	6.66
VGG [40] (v5)	6.8
PReLU-net [12]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

ResNet 앙상블이 top-5 에러 3.57%를 달성하며 ILSVRC 2015 대회에서 1위를 차지했다.

Deeper Bottleneck Architectures



152층 같은 깊은 네트워크를 기본 블록(3x3, 3x3) 그대로 쌓으면 연산량이 매우 커진다. 이를 방지하기 위해 사용된 것이 bottleneck이다. Bottleneck에서는 층 개수가 3개이지만 (기본 블록에서는 2개) 채널을 줄인 채로 3x3 연산을 하기 때문에 전체 연산 비용은 기본 블록과 비슷한 수준으로 유지된다.

오른쪽의 identity shortcut을 projection shortcut으로 바꾸면 시간 복잡도와 모델 크기가 2배로 늘어난다. projection은 256 채널 → 256 채널을 다루는 연산을 해야 하는데, 이는 1x1 conv(256 → 64 또는 64 → 256)보다 파라미터가 훨씬 많은 연산이다. 이 projection이 숏컷마다 하나씩 들어간다면 시간 복잡도와 모델 크기가 급격하게 커질 수밖에 없다.

구조

- F(잔차 함수)를 이루는 총 3개
- $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
- 첫 1×1 은 차원을 줄이는 역할을 한다. 3×3 은 실제 공간적 특징을 뽑아내는 핵심 연산을 수행한다. 마지막 1×1 은 차원을 다시 늘려서 복원하는 역할을 한다.

50-layer ResNet

- 34층 network에서 쓰던 2층짜리 기본 블록을 모두 3층짜리 bottleneck 블록으로 교체함
- 차원이 늘어나는 지점에 B 사용
- 3.8 billion FLOPs, 34층짜리 network와 큰 차이 없음. 층은 50층으로 늘어났지만 연산량은 34층짜리와 거의 비슷하게 유지됨.

101-layer and 152-layer ResNets

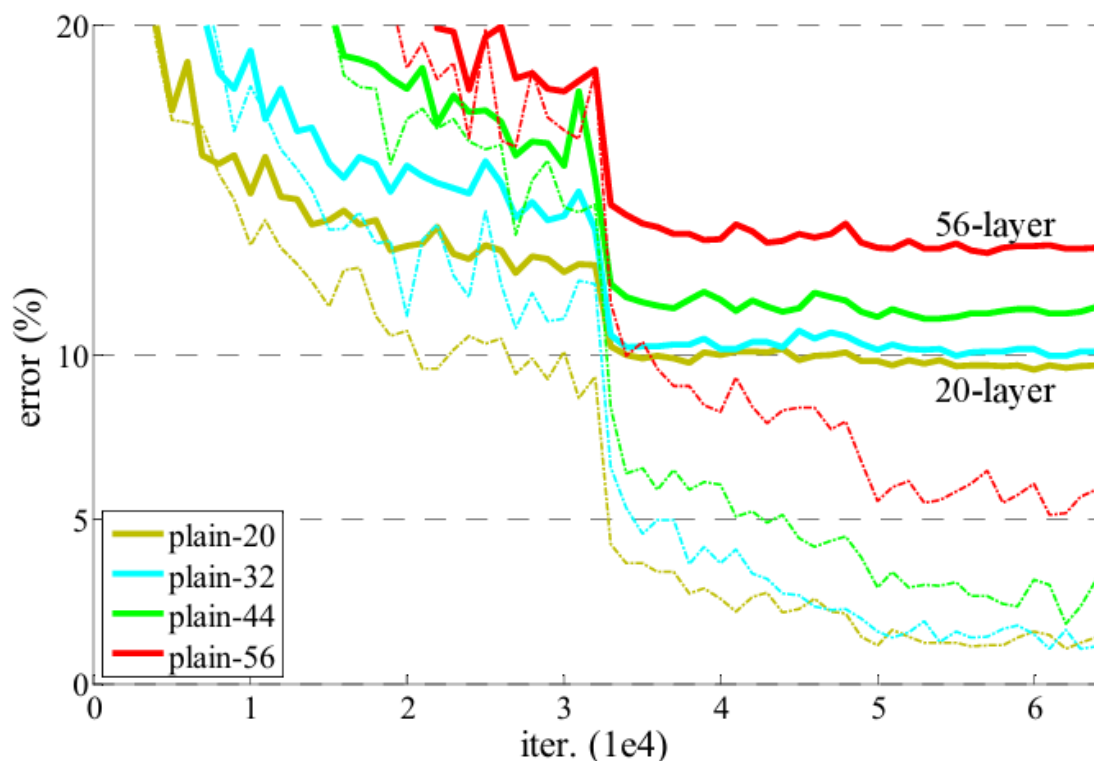
- 3층짜리 bottleneck 블록을 더 많이 쌓아서 만듦
- 11.3 billion FLOPs, VGG-16/19(15.3/19.6 billion FLOPs)보다 연산량 낮음
- 50/101/152층이 34층보다 훨씬 더 정확함
- 성능 저하 문제 관찰되지 않음
- 이 경향이 top-1, top-5 등 모든 평가 지표에서 일관되게 나타남

4.2. CIFAR-10 and Analysis

Dataset

- CIFAR-10
- 32×32 이미지 입력

Baseline



CIFAR-10에서 plain nets의 성능 저하가 재확인되었다. 깊어질수록 학습 에러가 높아지는 것을 확인할 수 있다. 이 최적화 어려움은 특정 데이터셋만의 문제가 아니라 fundamental 문제이다.

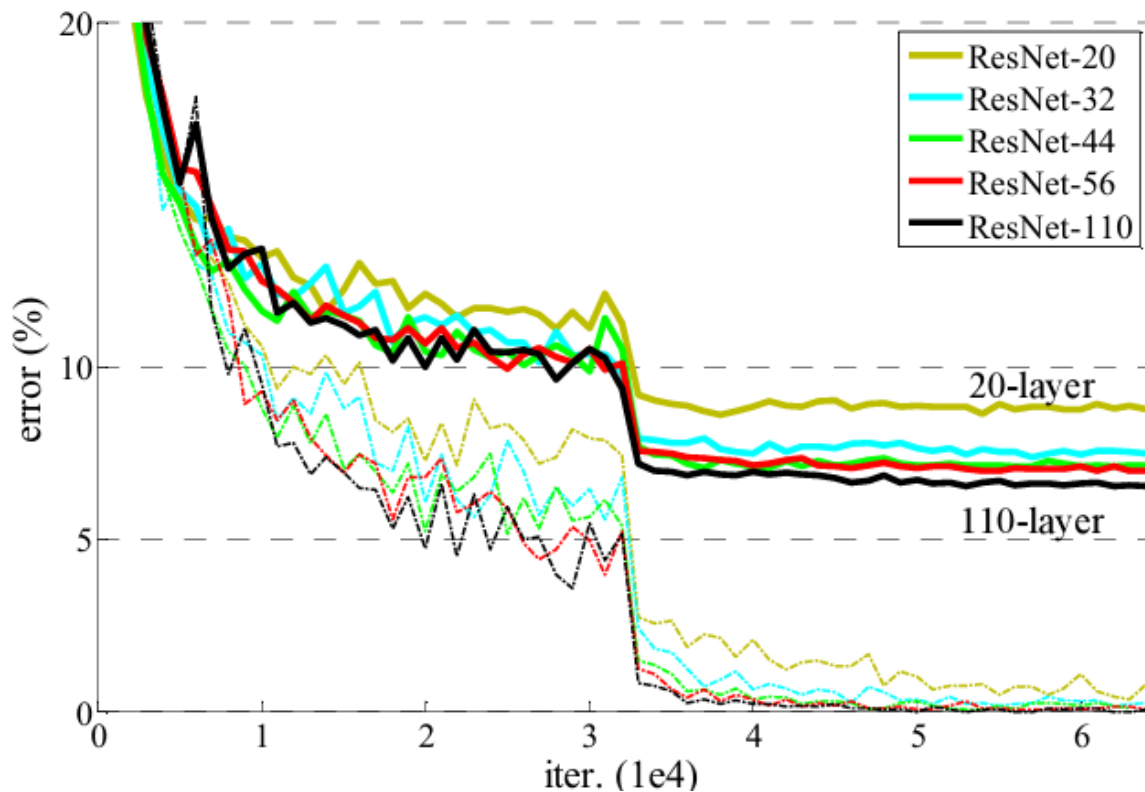
결과

구조

- 첫 레이어는 3x3 conv
- 그 다음 6n개의 3x3 conv 레이어를 쌓음. feature map 크기 {32, 16, 8} 각각에 대해 2n개씩 배정.
- 마지막엔 global average pooling → 10-way FC → softmax
- 총 레이어 수: $6n + 2$
- 슛컷은 3x3 레이어 쌍마다 연결(3n개), 전부 A 사용

method			error (%)
Maxout [9]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [34]	19	2.5M	8.39
Highway [41, 42]	19	2.3M	7.54 (7.72±0.16)
Highway [41, 42]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

ResNet-110이 6.43%로 가장 좋고 파라미터 수도 Highway 32층(1.25M)보다 조금 많지만 성능은 훨씬 좋았다. ResNet-1202의 에러율은 7.93%로 110층짜리보다 조금 더 나쁘게 나왔다. 과적합 가능성이 있는 것으로 보인다.

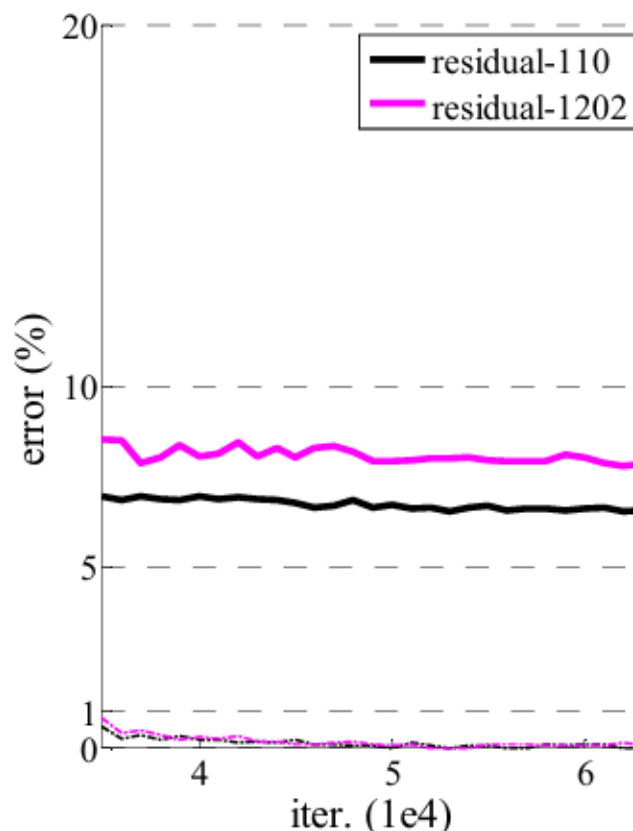


ResNet의 경우 깊어질수록 꾸준히 정확도가 개선됨을 확인했다. Plain net과는 정반대의 양상을 보인다. 처음부터 학습률 0.1로 시작하면 수렴이 잘 되지 않는다. 그래서 0.01로 시작한 다음 학습 에러가 80% 아래로 떨어질 때까지(약 400 iteration) 워밍업한 다음, 다시 0.1로 복귀해서 나머지 학습을 진행했다.

Analysis of Layer Responses

각 3x3 레이어의 출력을 BN 통과 직후, ReLU 적용 이전 시점에 측정했다. 이 분석은 잔차 함수의 응답 강도를 보여준다. 해당 값의 크기가 작다는 건 곧 잔차 함수가 작은 값을 낸다는 것을 직접적으로 나타낸다.

Exploring Over 1000 layers



1202층 네트워크에서 최적화 어려움은 전혀 없었다. 학습 에러 또한 0.1% 미만을 달성했다. 하지만 테스트 에러가 110층보다 나쁘게 나왔다. (7.93% vs 6.43%) 이는 과적합 때문일 가능성이 크다. 이는 1202층이나 되는 극도로 깊은 네트워크도 최적화 자체는 문제 없이 잘 된다는 것을 보여준다. 1202층짜리 모델은 CIFAR-10처럼 작은 데이터셋에 비해 불필요하게 크다.

5. 결론 (배운점)



연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

실험에서 detection 방법 자체와 구현 방식은 동일하게 유지하고 backbone만 VGG-16에서 ResNet-101로 바꾸었다. 즉 성능 개선은 오로지 backbone의 차이에서만 기인한다.

ResNet에서 검증 성능이 좋아진 이유는 학습 자체가 더 잘 되었기 때문이다. 잔차 학습은 극도로 깊은 시스템에서 효과가 있다.

ResNet 앙상블로 top-5 테스트 에러 3.57%를 달성했고 이 결과로 ILSVRC 2015 대회 1위를 차지하였다. 이밖에 COCO detection, COCO segmentation 트랙 등에서도 1위를 차지하였다.

ResNet은 범용적으로 다양한 태스크에 적용 가능하다.

다만 왜 항등 매핑이 최적일 때 잔차를 0으로 만드는 것이 $H(x)$ 를 직접 근사하는 것보다 더 쉬운지에 대한 완전한 이론적 증명이 부족했다.

파트 구성이 잘 나뉘어져 있어서 각 소제목별로 내용을 정리하기 좋았다. 수업 시간에만 배운 ResNet에 대해 꼼꼼하게 뜯어볼 수 있는 좋은 기회였다. 다만 결론 부분이 명시적으로 나타나 있지 않아 마무리가 조금 아쉬웠다는 생각이 든다.